

WEEK 2: PRODUCT AND CATEGORY MANAGEMENT SYSTEM

- ABINAYA.R (E25A1102)

Scenario

The e-commerce company wants to maintain information about different products and their categories. Each product belongs to a particular category. The system should store and manage details such as **Product Name, Price, and Stock Availability**.

1. Design Product and Category Tables

Category Table

The Category table is used to store information about different product categories. The **Category_ID** uniquely identifies each category.

```
SQL> CREATE TABLE Category1(Category_ID VARCHAR2(7),Category_Name VARCHAR2(20),Description VARCHAR2(50));  
Table created.
```

Product Table

The Product table is used to store details about the products. It contains information such as **Product_ID, Product_Name, Price, Stock, and Category_ID**.

```
SQL> CREATE TABLE Product1(Product_ID VARCHAR2(7),Product_Name VARCHAR2(20),Category_ID VARCHAR2(7),Price NUMBER(9,2),Stock_Quantity Number);  
Table created.
```

The two tables, **Category** and **Product**, have been created. Now, we need to define the relationship between the tables and apply the required constraints.

2. Define Relationships and Constraints

Relationship Between Category and Product

A category can contain many products, but each product belongs to only one category.

Therefore, the relationship between the two tables is:

One Category → Many Products

**Relationship: One-to-Many (1 :)*

Primary Key

A **Primary Key** is a column that uniquely identifies each record in a table. It does not allow duplicate values.

- **Category_ID** is the Primary Key in the Category table.

- **Product_ID** is the Primary Key in the Product table.

```
SQL> ALTER TABLE Category ADD CONSTRAINT PK_category PRIMARY KEY(category_ID);  
Table altered.
```

```
SQL> ALTER TABLE Product ADD CONSTRAINT PK_product PRIMARY KEY(product_ID);  
Table altered.
```

Foreign Key

A **Foreign Key** is used to connect two tables.

The **Category_ID** in the Product table acts as the Foreign Key. It refers to the **Category_ID** in the Category table.

```
SQL> ALTER TABLE product ADD CONSTRAINT FK_Product_Category FOREIGN KEY (Category_ID) REFERENCES Category(category_ID);  
Table altered.
```

NOT NULL Constraint

The **NOT NULL** constraint is used when a column must contain a value. It does not allow the field to be left empty.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;  
Table altered.  
  
SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(100) NOT NULL;  
Table altered.
```

CHECK Constraint

The **CHECK** constraint is used to make sure that the value entered satisfies a particular condition.

For example, the product price can be checked to make sure that it is greater than zero.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (PRICE > 0);  
Table altered.  
  
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE Constraint

The **UNIQUE** constraint is used to make sure that duplicate values are not entered into a column.

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);
Table altered.
```

3. Insert Values

Category Table

Values are inserted into the Category table using the **INSERT** command.

```
SQL> INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description');
Enter value for category_id: CAT101
Enter value for category_name: Electronics
Enter value for description: Electronic Gadgets and Devices
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('CAT101','Electronics','Electronic Gadgets and Devices')

1 row created.
```

```
SQL> /
Enter value for category_id: CAT102
Enter value for category_name: Clothing
Enter value for description: Men,Women and kids apparels
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('CAT102','Clothing','Men,Women and kids apparels')

1 row created.
```

```
SQL> /
Enter value for category_id: CAT103
Enter value for category_name: Grocery
Enter value for description: Food items and daily essentials
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('CAT103','Grocery','Food items and daily essentials')

1 row created.
```

```
SQL> /
Enter value for category_id: CAT104
Enter value for category_name: Furniture
Enter value for description: Home and office Furniture
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('CAT104','Furniture','Home and office Furniture')

1 row created.
```

```
SQL> /
Enter value for category_id: CAT105
Enter value for category_name: Books
Enter value for description: Educational and general Books
old 1: INSERT INTO Category VALUES('&Category_ID','&Category_Name','&Description')
new 1: INSERT INTO Category VALUES('CAT105','Books','Educational and general Books')

1 row created.
```

Product Table

Values are inserted into the Product table using the **INSERT** command.

```
SQL> INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity);
Enter value for product_id: P101
Enter value for product_name: Shoe
Enter value for category_id: CAT108
Enter value for price: 370
Enter value for stock_quantity: 4
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P101', 'Shoe', 'CAT108', 370, 4)

1 row created.

SQL> /
Enter value for product_id: P111
Enter value for product_name: Sketch
Enter value for category_id: CAT109
Enter value for price: 20
Enter value for stock_quantity: 10
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P111', 'Sketch', 'CAT109', 20, 10)

1 row created.

SQL> /
Enter value for product_id: P113
Enter value for product_name: Hair Dryer
Enter value for category_id: CAT101
Enter value for price: 1200
Enter value for stock_quantity: 3
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P113', 'Hair Dryer', 'CAT101', 1200, 3)

1 row created.

SQL> /
Enter value for product_id: P103
Enter value for product_name: Dinning Table
Enter value for category_id: CAT104
Enter value for price: 54000
Enter value for stock_quantity: 10
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P103', 'Dinning Table', 'CAT104', 54000, 10)

1 row created.
```

```
SQL> /
Enter value for product_id: P108
Enter value for product_name: Lipstick
Enter value for category_id: CAT107
Enter value for price: 290
Enter value for stock_quantity: 23
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P108', 'Lipstick', 'CAT107', 290, 23)

1 row created.

SQL> /
Enter value for product_id: P115
Enter value for product_name: TOOR DHALL
Enter value for category_id: CAT103
Enter value for price: 120
Enter value for stock_quantity: 19
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P115', 'TOOR DHALL', 'CAT103', 120, 19)

1 row created.

SQL> /
Enter value for product_id: P109
Enter value for product_name: Wings of fire
Enter value for category_id: CAT105
Enter value for price: 450
Enter value for stock_quantity: 3
old 1: INSERT INTO Product VALUES('&Product_ID', '&Product_Name', '&Category_ID', &Price, &Stock_Quantity)
new 1: INSERT INTO Product VALUES('P109', 'Wings of fire', 'CAT105', 450, 3)

1 row created.
```

Like this, insert a few more rows.

After Insertion – Category Table

```
SQL> INSERT INTO Product (Product_ID,Product_name,Category_ID,Price,Stock_Quantity)
VALUES ('P121','Cot','CAT103',19000,7);

1 row created.
```

4. CRUD Operations

CRUD stands for:

- **C – Create**
- **R – Read**
- **U – Update**
- **D – Delete**

CRUD operations are used to manage the records stored in the database.

Insert Record

A new product can be added to the Product table using the **INSERT** command.

```
SQL> INSERT INTO Product (Product_ID,Product_name,Category_ID,Price,Stock_Quantity)VALUES ('P121','Cot','CAT103',19000,7);

1 row created.
```

Update Price

The price of an existing product can be changed using the **UPDATE** command.

```
SQL> UPDATE Product SET Price = 56000 WHERE Product_Name = 'Dinning Table';

1 row updated.
```

Delete a Product

An existing product can be removed from the Product table using the **DELETE** command.

```
SQL> DELETE FROM Product WHERE Product_Name = 'Dinning Table';

1 row deleted.
```

5. Generate Category-wise Product Reports

1. Display all products under each category

This report is used to display all the products belonging to each category.

```
SQL> SELECT c.CATEGORY_ID,c.CATEGORY_NAME,p.PRODUCT_ID,p.PRICE,P.STOCK_QUANTITY FROM Category c JOIN Product p ON c.CATEGORY_ID = p.CATEGORY_ID ORDER BY c.CATEGORY_ID;
```

CATEGORY			
CATEGORY_NAME			
PRODOC	PRICE	STOCK_QUANTITY	
CAT101			
Electronics			
P113	1200		3
CAT103			
Grocery			
P115	120		19
CATEGORY			
CATEGORY_NAME			
PRODOC	PRICE	STOCK_QUANTITY	
CAT103			
Grocery			
P121	19000		7
CAT105			
Books			
CATEGORY			
CATEGORY_NAME			
PRODOC	PRICE	STOCK_QUANTITY	
CAT109			
	450		3
CAT107			
Beauty			
P108	290		23

2. Count the number of products in each category

This report shows how many products are available under each category.

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count FROM Product P JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	
PRODUCT_COUNT	
Footwear	1
Stationery	1
Books	1
CATEGORY_NAME	
PRODUCT_COUNT	
Beauty	1
Electronics	1
Grocery	2

6 rows selected.

3. Find the highest-priced product in each category

This report is used to find the product with the highest price in every category.

```
SQL> SELECT c.Category_Name,p.Product_Name,p.Price AS Highest_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID WHERE p.Price =(SELECT MAX(p2.Price)
OM Product p2 WHERE p2.Category_ID = p.Category_ID);
```

CATEGORY_NAME	PRODUCT_NAME	HIGHEST_PRICE
Footwear	Shoe	370
Stationery	Sketch	20
Electronics	Hair Dryer	1200
Beauty	Lipstick	290
Books	Wings of fire	

4. Display categories having more than 5 products

This report displays only those categories that contain more than five products.

```
SQL> SELECT c.Category_Name FROM Product P JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_name HAVING COUNT(p.Product_ID) > 5;
no rows selected
```

5. Find the average product price category-wise

This report calculates the average price of products available in each category.

```
SQL> SELECT c.Category_Name,AVG(p.Price) AS Avg_Price FROM Product p JOIN Category c ON p.Category_ID = c.Category_ID GROUP BY c.Category_Name;
```

CATEGORY_NAME	AVG_PRICE
Footwear	370
Stationery	20
Books	450
Beauty	290
Electronics	1200
Grocery	9560

6 rows selected.